Lab 7 - Activity 1

```java
package Lab7.Activities.Activity1;

public class Activity1
{
    public static void main(String[] args)
    {
        B b = new B();
    }
}

class A
{
    public A()
    {
        System.out.println("A's no-arg constructor is invoked");
    }
}

class B extends A {}

// OUTPUT:
// A's no-arg constructor is invoked
//
```

Program Output

```
A's no-arg constructor is invoked
```

Lab 7 - Activity 2

```java
package Lab7.Activities.Activity2;

public class Activity2
{
    public static void main(String[] args)
    {
        B b = new B(3);
    }
}

class A
{
    public A()
    {
        System.out.println("A's no-arg constructor is invoked");
    }
}

class B extends A
{
    public B(int t)
    {
        System.out.println("B's constructor is invoked");
    }
}

// OUTPUT:
// A's no-arg constructor is invoked
// B's constructor is invoked
//
```

Program Output

```
A's no-arg constructor is invoked
B's constructor is invoked
```

Lab 7 - Activity 3

```java
package Lab7.Activities.Activity3;

public class Activity3
{
    public static void main(String[] args)
    {
        B b = new B();
    }
}

class A
{
    public A(int x) {}
}

class B extends A
{
    /** Reason:
     * In java every child class is required to call the constructor of parent class
(super)
     * the call to super is not required to be explicit if super constructor is no
args
     * is super requires arguments, java doesn't know what to pass as arguments
     * therefore, we need to explicity tell java, to call with the given args
     */
    // public B() {}

    /**
     * Solution: We can either pass 0 to the `x` argument of super (constructor of
parent)
     * <br>
     * <br>
     * {@code public B() {super(0);}}
     * <br>
     * <br>
     * Or take value of x from the user, and then pass it
     * <br>
     * {@code public B(int x) {super(x);}}
     */
    public B() {super(0);}
}
```

Program Output

```
[No output captured - check if program ran correctly]
```

Lab 7 - Activity 4

```java
package Lab7.Activities;

public class Activity4
{
    public static void main(String[] args)
    {
        B b = new B(5, 10);
        System.out.println("Area = " + b.getArea());
    }
}

class Circle
{
    private double radius;

    public Circle(double radius)
    {
        // radius = radius; : ERROR: we need to use `this.`
        this.radius = radius;
    }

    public double getRadius()
    {
        return radius;
    }

    public double getArea()
    {
        return radius * radius * Math.PI;
    }
}

class B extends Circle
{
    private double length;

    // B(double radius, double length) : ERROR: constructor needs to be public to be
called in `main`
    public B(double radius, double length)
    {
        // Circle(radius); : ERROR: to call constructor of parent class, we use
`super`
        super(radius);
        // length = length; : ERROR: we need to use `this.`
        this.length = length;
    }

    /**
     * Override getArea()
     */
    public double getArea()
    {
```

```
        // return getArea() * length; : ERROR: this line will call itself
recursively, we need to use super
        return super.getArea() * length;
    }
}
```

Program Output

```
Area = 785.3981633974483
```

Lab 7 - Task 1

```java
package Lab7.Tasks;

import java.util.Date;
import java.util.Scanner;

public class Task1
{
    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args)
    {
        double width = getDouble("Enter width: ");
        double height = getDouble("Enter height: ");
        String color = getString("Enter color: ");
        boolean filled = getBoolean("Is filled (true/false): ");

        Rectangle rectangle = new Rectangle(width, height, color, filled);

        System.out.println("Area: " + rectangle.getArea());
        System.out.println("Perimeter: " + rectangle.getPerimeter());
        System.out.println("Color: " + rectangle.getColor());
        System.out.println("Filled: " + rectangle.isFilled());
    }

    private static boolean getBoolean(String message)
    {
        System.out.print(message);
        return scanner.nextBoolean();
    }

    private static String getString(String message)
    {
        scanner.nextLine(); // consume the newline character
        System.out.print(message);
        return scanner.nextLine();
    }

    private static double getDouble(String message)
    {
        System.out.print(message);
        return scanner.nextDouble();
    }
}

class GeometricObject
{
    String color;
    boolean filled;
    Date dateCreated;

    public GeometricObject()
    {
```

```
        this.color = "white";
        this.filled = true;
        this.dateCreated = new Date();
    }

    public GeometricObject(String color, boolean filled)
    {
        this.color = color;
        this.filled = filled;
        this.dateCreated = new Date();
    }

    @Override
    public String toString()
    {
        return this.getClass().getSimpleName() + ": color = " + color + ", filled =
" + filled;
    }

    /* GETTERS AND SETTERS */

    public String getColor()
    {
        return color;
    }

    public void setColor(String color)
    {
        this.color = color;
    }

    public boolean isFilled()
    {
        return filled;
    }

    public void setFilled(boolean filled)
    {
        this.filled = filled;
    }

    public Date getDateCreated()
    {
        return dateCreated;
    }
}

class Rectangle extends GeometricObject
{
    double width;
    double height;

    public Rectangle()
    {
```

```
        this.width = 0;
        this.height = 0;
    }

    public Rectangle(double width, double height)
    {
        this.width = width;
        this.height = height;
    }

    public Rectangle(double width, double height, String color, boolean filled)
    {
        super(color, filled);
        this.width = width;
        this.height = height;
    }

    public double getArea()
    {
        return width * height;
    }

    public double getPerimeter()
    {
        return 2 * (width + height);
    }

    @Override
    public String toString()
    {
        return "Rectangle: width = " + width + " height = " + height;
    }

    /* GETTERS AND SETTERS */

    public double getWidth()
    {
        return width;
    }

    public void setWidth(double width)
    {
        this.width = width;
    }

    public double getHeight()
    {
        return height;
    }

    public void setHeight(double height)
    {
        this.height = height;
```

```
    }
}
```

Program Output

```
Enter width: 10
Enter height: 10
Enter color: yellow
Is filled (true/false): true
Area: 100.0
Perimeter: 40.0
Color: yellow
Filled: true
```